NAME: Yash Lamba
Roll Number: 2401201038

Input :

```
# ============================================================
#  OS Lab Assignment 1 — CPU Scheduling: FCFS & SJF
#  Course: ENCA252 | Program: BCA H (AI & DS) | Name: Yash Lamba
# ============================================================

# ============================================================
# TASK 1: Process Class & Input Handling
# ============================================================

# A class is like a blueprint. Here we define what a "Process" looks like.
class Process:
    def __init__(self, pid, at, bt):
        """
        pid = Process ID       (e.g., P1, P2 ...)
        at  = Arrival Time     (when the process arrives in the ready
queue)
        bt  = Burst Time       (how long it needs on the CPU)
        """
        self.pid = pid
        self.at  = at
        self.bt  = bt

        # These will be calculated later by the scheduling algorithms
        self.ct  = 0   # Completion Time
        self.tat = 0   # Turnaround Time = CT - AT
        self.wt  = 0   # Waiting Time    = TAT - BT


def get_processes():
    """Ask the user to enter process details and return a list of Process
objects."""

    print("\n" + "="*55)
    print("       CPU SCHEDULING SIMULATOR")
    print("="*55)
```

```
    # Ask how many processes the user wants to enter (must be at least 4)
    while True:
        try:
            n = int(input("\nEnter number of processes (min 4): "))
            if n < 4:
                print("  ⚠  Please enter at least 4 processes.")
            else:
                break
        except ValueError:
            print("  ⚠  Invalid input. Please enter a number.")

    processes = []  # Empty list; we'll fill it with Process objects

    print(f"\nEnter Arrival Time and Burst Time for each process:")
    print("-" * 45)

    for i in range(1, n + 1):
        while True:
            try:
                at = int(input(f"  P{i} → Arrival Time: "))
                bt = int(input(f"  P{i} → Burst Time:   "))
                if at < 0 or bt <= 0:
                    print("  ⚠  Times must be non-negative (Burst Time > 0).")
                    continue
                processes.append(Process(f"P{i}", at, bt))
                print()
                break
            except ValueError:
                print("  ⚠  Invalid input. Enter whole numbers only.")

    return processes


def print_table(processes, title="Scheduling Results"):
    """Print any list of processes in a neat formatted table."""

    print("\n" + "="*55)
    print(f"  {title}")
```

```
    print("="*55)
    print(f"  {'PID':<6} {'AT':<6} {'BT':<6} {'CT':<6} {'TAT':<6}
{'WT':<6}")
    print("-" * 45)

    for p in processes:
        print(f"  {p.pid:<6} {p.at:<6} {p.bt:<6} {p.ct:<6} {p.tat:<6}
{p.wt:<6}")

    print("-" * 45)


# ============================================================
# TASK 2: FCFS Scheduling (First Come First Serve)
# ============================================================

def fcfs(processes):
    """
    FCFS: The process that arrives FIRST gets the CPU first.
    It is NON-PREEMPTIVE — once a process starts, it runs to completion.

    Steps:
      1. Sort processes by Arrival Time (AT)
      2. Track current time; handle idle gaps
      3. Calculate CT, TAT, WT for each process
    """

    # We sort by arrival time — whoever comes first, runs first
    # We use 'key=lambda p: p.at' which means "sort by the .at attribute"
    sorted_procs = sorted(processes, key=lambda p: p.at)

    current_time = 0        # The "clock" of the CPU
    gantt = []              # Will store (process_name, start_time,
end_time)

    for p in sorted_procs:

        # If CPU is idle (current process hasn't arrived yet),
fast-forward time
        if current_time < p.at:
```

```
            gantt.append(("IDLE", current_time, p.at))  # Record idle
period
            current_time = p.at

        start_time = current_time           # Process starts running now

        current_time += p.bt                # Process runs for its burst
time

        p.ct  = current_time                # It finishes at current_time
        p.tat = p.ct - p.at                 # Total time from arrival to
finish
        p.wt  = p.tat - p.bt                # Time it spent waiting

        gantt.append((p.pid, start_time, p.ct))

    return sorted_procs, gantt


# ============================================================
# TASK 3: SJF Scheduling (Shortest Job First — Non-Preemptive)
# ============================================================

def sjf(processes):
    """
    SJF: Among all ARRIVED processes, pick the one with the SMALLEST burst
time.
    It is NON-PREEMPTIVE — once chosen, it runs to completion.

    Steps:
      1. At each step, look at all processes that have arrived so far
      2. From those, pick the one with minimum Burst Time
      3. Run it fully, then repeat
    """

    # We work with copies so we don't mess up the original list
    import copy
    remaining = copy.deepcopy(processes)

    # Reset any previous scheduling results on original objects
```

```
    for p in processes:
        p.ct = p.tat = p.wt = 0

    current_time = 0
    completed    = []      # Processes that have finished
    gantt        = []      # Gantt chart data

    while remaining:

        # Find all processes that have ALREADY arrived by current_time
        available = [p for p in remaining if p.at <= current_time]

        if not available:
            # No process has arrived yet — CPU is idle
            # Jump time to the nearest arrival
            next_arrival = min(remaining, key=lambda p: p.at).at
            gantt.append(("IDLE", current_time, next_arrival))
            current_time = next_arrival
            continue

        # From available processes, pick the one with SHORTEST burst time
        # If two have equal BT, pick the one that arrived first (stable
sort)
        shortest = min(available, key=lambda p: (p.bt, p.at))

        start_time   = current_time
        current_time += shortest.bt

        shortest.ct  = current_time
        shortest.tat = shortest.ct - shortest.at
        shortest.wt  = shortest.tat - shortest.bt

        gantt.append((shortest.pid, start_time, shortest.ct))

        completed.append(shortest)
        remaining.remove(shortest)   # This process is done; remove from
queue

    # Now copy results back to the original process objects (matched by
PID)
```

```
    for orig in processes:
        for done in completed:
            if orig.pid == done.pid:
                orig.ct  = done.ct
                orig.tat = done.tat
                orig.wt  = done.wt

    return completed, gantt


# ============================================================
# TASK 4: Gantt Chart (Text-Based Visualization)
# ============================================================

def print_gantt_chart(gantt, title="Gantt Chart"):
    """
    Draw a simple text-based Gantt chart showing execution order.

    Example output:
    | P1 | P3 | IDLE | P2 |
    0    3    7     10   14
    """

    print(f"\n  {title}")
    print("  " + "-" * (len(gantt) * 8))

    # Top row: process names in boxes
    row = "  |"
    for (name, start, end) in gantt:
        label = name.center(6)    # Center the name in a 6-char wide cell
        row += f" {label} |"
    print(row)

    print("  " + "-" * (len(gantt) * 8))

    # Bottom row: time markers
    time_row = "  "
    for (name, start, end) in gantt:
        time_row += str(start).ljust(8)    # Left-justify start times
    # Add the very last end time
```

```
    time_row += str(gantt[-1][2])
    print(time_row)



# ============================================================
# TASK 5: Performance Analysis & Comparison
# ============================================================

def performance_analysis(fcfs_procs, sjf_procs):
    """
    Calculate and compare average WT and TAT for both algorithms.
    Then give a conclusion on which is better and why.
    """

    n = len(fcfs_procs)

    # Calculate averages for FCFS
    fcfs_avg_tat = sum(p.tat for p in fcfs_procs) / n
    fcfs_avg_wt  = sum(p.wt  for p in fcfs_procs) / n

    # Calculate averages for SJF
    sjf_avg_tat  = sum(p.tat for p in sjf_procs) / n
    sjf_avg_wt   = sum(p.wt  for p in sjf_procs) / n

    print("\n" + "="*55)
    print("  PERFORMANCE COMPARISON: FCFS vs SJF")
    print("="*55)
    print(f"  {'Metric':<30} {'FCFS':<10} {'SJF':<10}")
    print("-" * 50)
    print(f"  {'Average Turnaround Time (TAT)':<30} {fcfs_avg_tat:<10.2f}
{sjf_avg_tat:<10.2f}")
    print(f"  {'Average Waiting Time (WT)':<30} {fcfs_avg_wt:<10.2f}
{sjf_avg_wt:<10.2f}")
    print("-" * 50)

    # Conclusion
    print("\n  📊 ANALYSIS:")
    if sjf_avg_wt < fcfs_avg_wt:
        print("  ✅ SJF has LOWER average waiting time.")
        print("     → SJF is better for minimizing wait time.")
```

```
        print("     → It reduces idle CPU time by running short jobs
first.")
    elif fcfs_avg_wt < sjf_avg_wt:
        print("  ✅ FCFS has LOWER average waiting time in this case.")
        print("     → This can happen when short jobs arrive late.")
    else:
        print("  Both algorithms performed equally in this case.")

    print("\n  ℹ️  TRADE-OFFS:")
    print("  • FCFS is simple but can cause the 'Convoy Effect'")
    print("    (long jobs blocking short ones).")
    print("  • SJF minimizes average waiting time but requires")
    print("    knowing burst times in advance (not always practical).")
    print("="*55)


# ============================================================
# MAIN — Runs everything
# ============================================================

def main():

    # ---------- Task 1: Get process input ----------
    processes = get_processes()

    # Show the input table
    print("\n" + "="*55)
    print("  INPUT: PROCESS TABLE")
    print("="*55)
    print(f"  {'PID':<6} {'Arrival Time':<15} {'Burst Time':<12}")
    print("-" * 40)
    for p in processes:
        print(f"  {p.pid:<6} {p.at:<15} {p.bt:<12}")
    print("-" * 40)

    # ---------- Task 2: Run FCFS ----------
    import copy
    fcfs_procs_input = copy.deepcopy(processes)   # Fresh copy for FCFS
    sjf_procs_input  = copy.deepcopy(processes)   # Fresh copy for SJF
```

```
    fcfs_result, fcfs_gantt = fcfs(fcfs_procs_input)
    print_table(fcfs_result, title="FCFS — Results")

    # ---------- Task 3: Run SJF ----------
    sjf_result, sjf_gantt = sjf(sjf_procs_input)
    print_table(sjf_result, title="SJF — Results")

    # ---------- Task 4: Gantt Charts ----------
    print("\n" + "="*55)
    print("  TASK 4 — GANTT CHARTS")
    print("="*55)
    print_gantt_chart(fcfs_gantt, title="FCFS Gantt Chart")
    print_gantt_chart(sjf_gantt,  title="SJF Gantt Chart")

    # ---------- Task 5: Performance Analysis ----------
    performance_analysis(fcfs_result, sjf_result)

    print("\n  ✅ Simulation Complete!\n")


# This line means: only run main() if THIS file is executed directly
# (not if it's imported by another script)
if __name__ == "__main__":
    main()
```

Output:

```
============================================================
       CPU SCHEDULING SIMULATOR
============================================================

Enter number of processes (min 4): 4

Enter Arrival Time and Burst Time for each process:
------------------------------------------------
  P1 → Arrival Time: 0
  P1 → Burst Time:   3

  P2 → Arrival Time: 1
  P2 → Burst Time:   5

  P3 → Arrival Time: 2
  P3 → Burst Time:   7

  P4 → Arrival Time: 3
  P4 → Burst Time:   9


============================================================
  INPUT: PROCESS TABLE
============================================================
  PID    Arrival Time    Burst Time
----------------------------------------
  P1     0               3
  P2     1               5
  P3     2               7
  P4     3               9
----------------------------------------
```

```
==================================================
  FCFS — Results
==================================================
  PID    AT     BT     CT     TAT    WT
------------------------------------------
  P1     0      3      3      3      0
  P2     1      5      8      7      2
  P3     2      7      15     13     6
  P4     3      9      24     21     12
------------------------------------------

==================================================
  SJF — Results
==================================================
  PID    AT     BT     CT     TAT    WT
------------------------------------------
  P1     0      3      3      3      0
  P2     1      5      8      7      2
  P3     2      7      15     13     6
  P4     3      9      24     21     12
------------------------------------------
```

```
==================================================
  FCFS — Results
==================================================
  PID    AT     BT     CT     TAT    WT
------------------------------------------
  P1     0      3      3      3      0
  P2     1      5      8      7      2
  P3     2      7      15     13     6
  P4     3      9      24     21     12
------------------------------------------

==================================================
  SJF — Results
==================================================
  PID    AT     BT     CT     TAT    WT
------------------------------------------
  P1     0      3      3      3      0
  P2     1      5      8      7      2
  P3     2      7      15     13     6
  P4     3      9      24     21     12
------------------------------------------
```

```
==================================================
  TASK 4 — GANTT CHARTS
==================================================

  FCFS Gantt Chart
  --------------------------------
  |   P1   |   P2   |   P3   |   P4   |
  --------------------------------
  0        3        8        15       24

  SJF Gantt Chart
  --------------------------------
  |   P1   |   P2   |   P3   |   P4   |
  --------------------------------
  0        3        8        15       24
```

```
==================================================
  PERFORMANCE COMPARISON: FCFS vs SJF
==================================================
  Metric                          FCFS       SJF
--------------------------------------------------
  Average Turnaround Time (TAT)  11.00      11.00
  Average Waiting Time (WT)      5.00       5.00
--------------------------------------------------

  📊 ANALYSIS:
  Both algorithms performed equally in this case.

  ℹ  TRADE-OFFS:
  • FCFS is simple but can cause the 'Convoy Effect'
    (long jobs blocking short ones).
  • SJF minimizes average waiting time but requires
    knowing burst times in advance (not always practical).
==================================================

  ✅ Simulation Complete!
```